

TECNOLOGIAS INALAMBRICAS

Práctica 3

Comunicación ESP32 con Base de Datos SQL Server en Docker mediante API REST con Node.js

Integrantes:

Eduardo Cadengo Lopez
Itzel Citlalli Martell De La Cruz
Damian Alexander Diaz Pina

25 de mayo de 2026

Índice

1. Introducción	3
1.1 Diagrama	3
2.1 ESP32	3
2.2 Docker y Contenedores	4
2.3 Microsoft SQL Server en Docker	4
Nota sobre la imagen utilizada	4
2.4 Node.js y API REST	4
2.5 Protocolo HTTP y formato JSON	5
3. Desarrollo de la Práctica	6
3.1 Creación del Contenedor Docker con SQL Server	6
3.2 Servidor API REST con Node.js	6
3.3 Programación del ESP32	6
3.4 Código IDE ESP32	7
4.1 Tabla de configuración general	12
4.2 Esquema de la tabla SQL	13
5. Resultados Obtenidos	13
6. Problemas Encontrados y Soluciones	14
6.1 Módulos Node.js no encontrados	14
6.2 Corrupción del contenedor azure-sql-edge	14
6.3 Inconsistencias en el archivo de configuración	14
6.4 Base de datos no existente	14
7. Conclusiones	15
8. Recursos del Proyecto	15

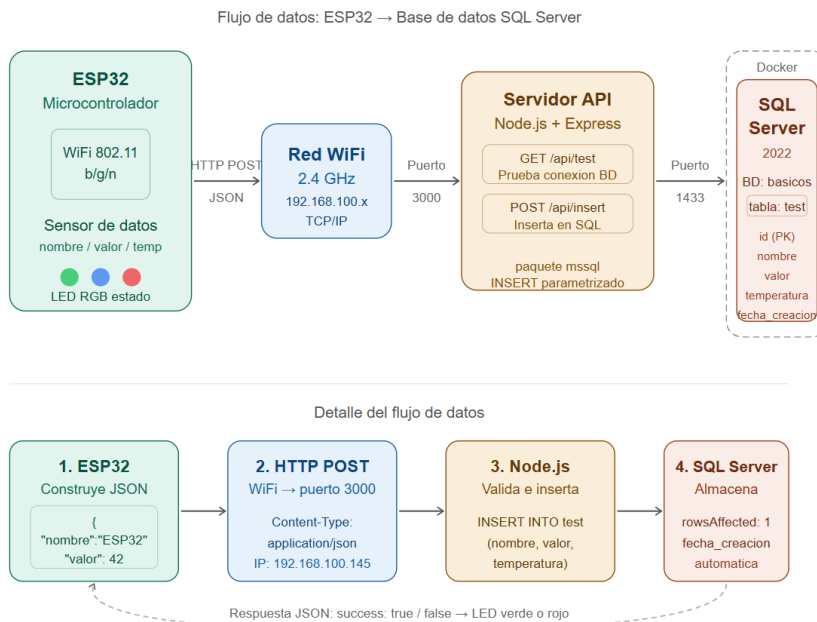
1. Introducción

En la actualidad, los sistemas embebidos tienen la capacidad de interactuar directamente con infraestructuras de datos empresariales a través de redes inalámbricas. Esta práctica explora precisamente ese escenario: un microcontrolador ESP32 envía información a una base de datos SQL Server que corre dentro de un contenedor Docker, usando como intermediario un servidor REST desarrollado con Node.js.

El objetivo central es comprender el flujo completo de los datos desde el hardware hasta el almacenamiento persistente, identificando cada capa del sistema: la capa de adquisición (ESP32), la capa de transporte (WiFi + HTTP), la capa de lógica de negocio (API Node.js) y la capa de persistencia (SQL Server en Docker).

Esta arquitectura es útil en contextos de IoT industrial, telemetría, monitoreo de sensores y cualquier aplicación donde sea necesario centralizar datos provenientes de múltiples dispositivos físicos.

1.1 Diagrama



2. Marco Teórico e Investigación

2.1 ESP32

El ESP32 es un microcontrolador de doble núcleo desarrollado por Espressif Systems que integra conectividad WiFi 802.11 b/g/n y Bluetooth 4.2 en un solo chip. Opera a frecuencias de hasta 240 MHz y cuenta con 520 KB de SRAM interna. Su arquitectura lo hace ideal para proyectos de Internet de las Cosas (IoT) ya que elimina la necesidad de módulos de comunicación inalámbrica adicionales.

Entre sus características más relevantes para esta práctica se encuentran la pila TCP/IP completa integrada, soporte para protocolos HTTP y HTTPS, cliente WiFi (modo STA) capaz de

conectarse a redes existentes, y compatibilidad con el entorno de desarrollo Arduino IDE mediante la instalación del paquete de soporte de placas ESP32.

La librería HTTPClient disponible para ESP32 permite realizar peticiones HTTP de tipo GET y POST de manera sencilla, lo que facilita la comunicación con APIs REST. También incluye la librería ArduinoJson, que permite serializar y deserializar datos en formato JSON, estándar ampliamente utilizado en servicios web modernos.

2.2 Docker y Contenedores

Docker es una plataforma de virtualización a nivel de sistema operativo que permite empaquetar aplicaciones y sus dependencias en unidades llamadas contenedores. A diferencia de las máquinas virtuales, los contenedores comparten el kernel del sistema operativo anfitrión, lo que los hace significativamente más ligeros y rápidos de iniciar.

Un contenedor Docker encapsula todo lo necesario para ejecutar una aplicación: código, runtime, herramientas del sistema, librerías y configuraciones. Esto garantiza que el comportamiento sea idéntico independientemente del entorno donde se despliegue, resolviendo el clásico problema de funciona en mi máquina pero no en el servidor.

Para esta práctica se utilizó Docker Desktop en Windows como plataforma de gestión de contenedores. La imagen de SQL Server se descargó del repositorio oficial de Microsoft y se configuró mediante variables de entorno al momento de crear el contenedor.

2.3 Microsoft SQL Server en Docker

Microsoft SQL Server es un sistema de gestión de bases de datos relacional (RDBMS) desarrollado por Microsoft. Soporta el lenguaje SQL estándar y agrega extensiones propietarias como Transact-SQL (T-SQL). Es ampliamente utilizado en entornos empresariales por su robustez, escalabilidad y soporte para transacciones ACID.

En esta práctica se utilizó la imagen oficial de SQL Server 2022 disponible en Docker Hub bajo el nombre mssql/server:2022-latest. Esta imagen corre sobre Linux y expone el puerto estándar 1433 para aceptar conexiones de clientes externos. La contraseña del usuario administrador (sa) se configura mediante la variable de entorno MSSQL_SA_PASSWORD al momento de crear el contenedor.

Nota sobre la imagen utilizada

Inicialmente se intentó utilizar la imagen azure-sql-edge, que es una versión optimizada para dispositivos de borde. Sin embargo, durante el desarrollo de la práctica el contenedor original sufrió corrupción de datos, lo que impidió continuar utilizándolo. Al intentar crear un nuevo contenedor con la misma imagen, el sistema no permitió la reasignación del puerto 1433, posiblemente por conflictos con datos residuales del contenedor anterior.

Por esta razón se optó por utilizar la imagen mssql/server:2022-latest, que es la versión completa de SQL Server 2022 para Linux. La principal diferencia práctica es la ubicación de la herramienta sqlcmd: en azure-sql-edge se encuentra en /opt/mssql-tools/bin/sqlcmd, mientras que en mssql/server:2022-latest se ubica en /opt/mssql-tools18/bin/sqlcmd.

2.4 Node.js y API REST

Node.js es un entorno de ejecución de JavaScript del lado del servidor basado en el motor V8 de Google Chrome. Su modelo de entrada/salida no bloqueante y orientado a eventos lo hace

especialmente eficiente para aplicaciones que manejan múltiples conexiones concurrentes, como servidores web y APIs.

En esta práctica se utilizó el framework Express.js para construir el servidor API REST. Express simplifica la definición de rutas HTTP y el manejo de solicitudes y respuestas. El servidor actúa como intermediario entre el ESP32 y la base de datos: recibe las peticiones HTTP del microcontrolador, valida los datos, y ejecuta las operaciones correspondientes en SQL Server usando el paquete mssql.

La API expone dos endpoints principales: uno de prueba de conexión (GET /api/test) y uno para insertar registros (POST /api/insert). Esta separación permite verificar la conectividad con la base de datos de forma independiente a la inserción de datos.

2.5 Protocolo HTTP y formato JSON

HTTP (HyperText Transfer Protocol) es el protocolo de comunicación estándar de la web. En su versión más básica define métodos como GET (obtener datos) y POST (enviar datos). El ESP32 utiliza estos métodos para comunicarse con el servidor Node.js a través de la red WiFi local.

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos basado en texto. Para esta práctica, el ESP32 serializa los datos del sensor en formato JSON antes de enviarlos al servidor, y el servidor los deserializa para insertarlos en la base de datos.

3. Desarrollo de la Práctica

3.1 Creación del Contenedor Docker con SQL Server

El primer paso fue crear el contenedor de base de datos ejecutando el siguiente comando en PowerShell con privilegios de administrador:

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=C0NTR453N1!4"  
-p 1433:1433 --name pruebaDb --hostname pruebaDB  
-d mssql/server:2022-latest
```

Los parámetros utilizados tienen el siguiente significado:

- ACCEPT_EULA=Y: acepta el acuerdo de licencia de SQL Server, requerido para ejecutar la imagen.
- MSSQL_SA_PASSWORD: establece la contraseña del usuario administrador sa.
- -p 1433:1433: mapea el puerto del contenedor al puerto del sistema anfitrión.
- --name pruebaDb: asigna un nombre al contenedor para referenciarlo fácilmente.
- -d: ejecuta el contenedor en modo desconectado (background).

Una vez activo el contenedor, se accedió a él mediante la pestaña Exec de Docker Desktop para crear la base de datos y la tabla usando sqlcmd:

```
/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "C0NTR453N1!4" -C  
1> create database basicos;  
2> go  
1> USE basicos;  
2> go  
1> CREATE TABLE test (id INT PRIMARY KEY IDENTITY(1,1),  
    nombre VARCHAR(50), valor INT, temperatura FLOAT,  
    fecha_creacion DATETIME DEFAULT GETDATE());  
2> go
```

3.2 Servidor API REST con Node.js

Se desarrolló un servidor intermediario en Node.js utilizando Express. Este servidor escucha en el puerto 3000 y expone dos rutas HTTP. La configuración de conexión a SQL Server incluye la IP del equipo en la red local, las credenciales del usuario sa y el nombre de la base de datos creada previamente.

El endpoint POST /api/insert recibe un cuerpo JSON con los campos nombre, valor y temperatura, establece una conexión al servidor SQL, ejecuta un INSERT parametrizado para prevenir inyecciones SQL, y responde con un objeto JSON indicando el resultado. Para iniciar el servidor se ejecutó:

```
npm install  
node servidor_api_esp32.js
```

3.3 Programación del ESP32

El código del ESP32 se estructuró para realizar las siguientes acciones en secuencia: conectarse a la red WiFi configurada, verificar la conexión con la base de datos mediante el endpoint de prueba, y realizar inserciones periódicas de datos cada 30 segundos.

La librería HTTPClient maneja las peticiones HTTP y ArduinoJson construye el cuerpo de las solicitudes POST. El LED RGB indica el estado del sistema: azul durante la conexión, verde al completar un INSERT exitoso, y rojo en caso de error.

3.4 Código IDE ESP32

```
// Código para ESP32 con leds o sin leds, o con sensor y sin sensor
// - Conexión a MSSQL vía servidor
// Librerías necesarias:
// - WiFi (incluida en Arduino IDE para ESP32)
// - HTTPClient (incluida en Arduino IDE para ESP32)

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// ===== CONFIGURACIÓN =====
const char* ssid = "Totalplay"; // Nombre del internet al que estes conectado
const char* password = "Contraseña"; // La contraseña del internet al que estes conectado
const char* apiUri = "http://192.168.100.145:3000/api"; // la ip y las demas
tambien es primero la ip del servidor o del internet al que este conectado
const char* testUri = "http://192.168.100.145:3000/api/test";
const char* insertUri = "http://192.168.100.145:3000/api/insert";

// Pines de trabajo del ESP32 S3 Ajusta estos valores según tu proyecto

// Pines de trabajo del ESP32 S3 Ajusta estos valores según tu proyecto

// Variables de control
unsigned long lastBlink = 0;
bool blinkState = false;
const int BLINK_INTERVAL = 500; // 500ms para parpadeo

// ===== FUNCIONES LED INTEGRADO=====
void setupLED() {
#define LED_PIN 48
    // Apagar todos los LEDs inicialmente
    digitalWrite(LED_PIN, LOW); // Turn the RGB LED off
}

void setLEDColor(String color) {
    if (color == "off") {
        rgbLedWrite(LED_PIN, 0, 0, 0);
    }
}
```

```

    }
    else if (color == "red") {
        rgbLedWrite(LED_PIN, RGB_BRIGHTNESS, 0, 0); // Red
    }
    else if (color == "green") {
        rgbLedWrite(LED_PIN, 0, RGB_BRIGHTNESS, 0); //Green
    }
    else if (color == "blue") {
        rgbLedWrite(LED_PIN, 0, 0, RGB_BRIGHTNESS); //Blue
    }
}

```

```

void blinkLED(String color) {
    unsigned long currentTime = millis();
    if (currentTime - lastBlink >= BLINK_INTERVAL) {
        lastBlink = currentTime;
        blinkState = !blinkState;

        if (blinkState) {
            setLEDColour(color);
        } else {
            setLEDColour("off");
        }
    }
}

```

```

// ===== FUNCIONES WiFi =====

```

```

void connectToWiFi() {
    Serial.println("\n\nInicializando conexión WiFi...");
    Serial.print("Conectando a: ");
    Serial.println(ssid);

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    int attemp = 0;
    while (WiFi.status() != WL_CONNECTED && attemp < 20) {
        delay(500);
        Serial.print(".");
        attemp++;

        // Parpadeo azul mientras se conecta
        blinkLED("blue");
    }
}

```



```

if (WiFi.status() == WL_CONNECTED) {
    setLEDColour("blue");
    Serial.println("\n✓ Conectado a WiFi");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
} else {
    setLEDColour("red");
    Serial.println("\nX Error: No se pudo conectar a WiFi");
}
}

// ===== FUNCIONES HTTP =====

bool testDatabaseConnection() {
    Serial.println("\nProbando conexión a MSSQL...");

    HTTPClient http;
    http.begin(testUrl);
    http.addHeader("Content-Type", "application/json");

    int httpCode = http.GET();

    if (httpCode == 200) {
        String response = http.getString();
        Serial.println("✓ Conexión a BD exitosa");
        Serial.println(response);
        http.end();
        return true;
    } else {
        Serial.print("X Error conectando a BD: ");
        Serial.println(httpCode);
        http.end();
        return false;
    }
}

bool insertDataToDatabase(const char* nombre, int valor, float temperatura) {
    Serial.println("\nEnviando INSERT a MSSQL...");

    HTTPClient http;
    http.begin(insertUrl);
    http.addHeader("Content-Type", "application/json");

    // Crear JSON con los datos

```

```

DynamicJsonDocument doc(200);
doc["nombre"] = nombre;
doc["valor"] = valor;
doc["temperatura"] = temperatura;

String jsonString;
serializeJson(doc, jsonString);

Serial.print("Datos enviados: ");
Serial.println(jsonString);

int httpCode = http.POST(jsonString);

if (httpCode == 200) {
    String response = http.getString();
    Serial.println("✓ INSERT exitoso");
    Serial.println(response);
    http.end();
    return true;
} else {
    Serial.print("X Error en INSERT: ");
    Serial.println(httpCode);
    String response = http.getString();
    Serial.println(response);
    http.end();
    return false;
}
}

// ===== SETUP =====

void setup() {

    Serial.begin(115200);
    delay(1000);

    Serial.println("\n\n=== ESP32 S3 MSSQL DEBUG ===");

    setupLED();

    // Conectar a WiFi

```

```

connectToWiFi();

if (WiFi.status() != WL_CONNECTED) {
    setLEDColor("red");
    Serial.println("No se pudo conectar a WiFi. Reiniciando en 5 segundos...");
    delay(5000);
    ESP.restart();
}

// Parpadeo azul mientras se prueba conexión a BD
Serial.println("\nProbando conexión a base de datos...");
bool connectionOk = false;

for (int i = 0; i < 10; i++) {
    blinkLED("blue");

    if (testDatabaseConnection()) {
        connectionOk = true;
        break;
    }
    delay(500);
}

if (!connectionOk) {
    setLEDColor("red");
    Serial.println("No se pudo conectar a la BD. Verifique la configuración.");
    delay(5000);
    ESP.restart();
}

// Hacer un INSERT de prueba
Serial.println("\nRealizando INSERT de prueba...");

if (insertDataToDatabase("ESP32_S3_Test", 42, 23.5)) {
    // INSERT exitoso - LED verde
    setLEDColor("green");
    Serial.println("\n✓✓✓ TODO CORRECTO - LED VERDE ✓✓✓");
} else {
    // INSERT fallido - LED rojo
    setLEDColor("red");
    Serial.println("\nXXX ERROR EN INSERT - LED ROJO XXX");
}
}

// ===== LOOP =====

```

```

void loop() {
    // Mantener el LED en el estado actual
    // El color ya está seteado (verde o rojo)

    // Cada 30 segundos, hacer otro INSERT para demostración
    static unsigned long lastInsert = 0;
    if (millis() - lastInsert > 30000) {
        lastInsert = millis();

        Serial.println("\n\n=== Enviando INSERT periódico ===");

        int valorRandom = random(0, 100);
        float temperaturaRandom = 20.0 + (random(0, 50) / 10.0);

        if (insertDataToDatabase("ESP32_Periodic", valorRandom, temperaturaRandom)) {
            setLEDColour("green");
            Serial.println("✓ INSERT periódico exitoso");

        } else {
            setLEDColour("red");
            Serial.println("X INSERT periódico fallido");

        }
    }

    delay(100);
    setLEDColour("off");
}

```

4. Configuración del Sistema

4.1 Tabla de configuración general

Componente	Configuracion
Red WiFi	Red local 2.4 GHz
IP del servidor(Depende de cual ip tengas)	192.168.100.145
Puerto API	3000
Puerto SQL Server (depende de cual puerto le hayas puesto)	1433

Base de datos (Depende de cual base de datos hayas creado)	basicos
Tabla SQL	test
Imagen Docker	mssql/server:2022-latest
Usuario BD	sa
Intervalo de inserción	30 segundos

4.2 Esquema de la tabla SQL

Campo	Tipo	Restriccion	Descripcion
id	INT	PK, IDENTITY	Identificador autoincremental
nombre	VARCHAR(50)		Nombre del dispositivo o sensor
valor	INT		Valor entero del sensor
temperatura	FLOAT		Valor de temperatura
fecha_creacion	DATETIME	DEFAULT GETDATE()	Marca de tiempo automática

5. Resultados Obtenidos

Al terminar la implementación de todos los componentes del sistema, se logró verificar el funcionamiento correcto de la cadena de comunicación completa. Los resultados obtenidos fueron los siguientes:

- El contenedor Docker con SQL Server 2022 inició correctamente y quedó accesible en el puerto 1433 de la red local.
- La base de datos basicos y la tabla test se crearon satisfactoriamente mediante sqlcmd desde la pestaña Exec de Docker Desktop.
- El servidor Node.js inició sin errores tras instalar las dependencias con npm install, y el endpoint GET /api/test respondió con un mensaje de conexión exitosa.
- El ESP32 se conectó a la red WiFi local, obtuvo una dirección IP asignada por el router, y envió correctamente las peticiones HTTP POST al servidor API.
- Los datos fueron insertados en la tabla SQL y se verificaron con un SELECT en sqlcmd, confirmando que los registros aparecen con los valores correctos y la marca de tiempo automática.

El LED RGB de la placa mostró correctamente los estados del sistema: parpadeo azul durante la fase de conexión y verde sostenido al confirmar inserciones exitosas.

6. Problemas Encontrados y Soluciones

6.1 Módulos Node.js no encontrados

Al intentar ejecutar el servidor por primera vez se obtuvo el error Cannot find module express. El problema era que los paquetes npm no estaban instalados localmente. La solución fue ejecutar npm install en el directorio del proyecto para descargar las dependencias listadas en package.json.

6.2 Corrupción del contenedor azure-sql-edge

Durante el desarrollo, el contenedor original basado en la imagen azure-sql-edge sufrió corrupción de datos. Al intentar recrearlo, SQL Server no aceptaba conexiones y el sistema reportaba conflictos en el puerto 1433. Se cambió a la imagen mssql/server:2022-latest. La principal diferencia técnica es la ruta de sqlcmd: /opt/mssql-tools18/bin/sqlcmd en lugar de /opt/mssql-tools/bin/sqlcmd.

6.3 Inconsistencias en el archivo de configuración

El archivo servidor_api_esp32.js contenía dos bloques de credenciales con contraseñas diferentes. Además, el nombre de la tabla en la consulta INSERT era prueba en lugar de test. Ambos problemas se corrigieron unificando las credenciales y actualizando el nombre de la tabla.

6.4 Base de datos no existente

Los logs de Docker mostraban el error 18456 con el mensaje Failed to open the explicitly specified database basics, lo que indicaba que la base de datos aún no había sido creada. Se resolvió accediendo al contenedor mediante la pestaña Exec de Docker Desktop y ejecutando los comandos CREATE DATABASE y CREATE TABLE en sqlcmd.

7. Conclusiones

Eduardo: Con esta práctica quedó claro cómo se integran tecnologías de distintos niveles: desde el firmware del ESP32 hasta una base de datos empresarial, usando una API REST como puente entre ambas capas.

Damian: Docker resultó muy práctico para levantar SQL Server sin tener que instalarlo directamente en el sistema. El contenedor asegura que el entorno funcione igual en cualquier equipo que tenga Docker.

Itzel: Usar una API REST como intermediario entre el hardware y la base de datos es una buena práctica en IoT: desacopla el firmware de la lógica de datos y permite que varios dispositivos usen el mismo punto de acceso. La combinación ESP32 + Node.js + Docker + SQL Server es una arquitectura viable, manejable y con buen potencial de escalar.

8. Recursos del Proyecto

Aquí se pueden consultar el código fuente del proyecto y el video de demostración con el funcionamiento completo del sistema:

- **Repositorio GitHub:**
https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/ESP32_DockerWiFi
- **Video de funcionamiento**
(YouTube): <https://youtube.com/shorts/i1MqJNeStbQ?feature=share>